Convergence Dynamics of Agentic Hyperparameter Optimization in Federated Learning

Jim Prantzalos
University of Michigan-Flint
jprantza@umich.edu

ABSTRACT

Federated learning enables collaborative model training across institutions without centralizing data, making it well-suited for healthcare applications. However, hyperparameter optimization in federated settings remains challenging due to limited observability, delayed feedback, and communication constraints. Existing approaches typically treat tuning as a fixed pre-training step, limiting the ability to adapt during training.

This study reframes hyperparameter optimization in federated learning as a sequential decision-making problem. We compare three strategies: a fixed baseline, static optimization using Optuna, and an agentic approach in which an LLM-driven controller proposes hyperparameters during training. Experiments are conducted on the eICU Collaborative Research Database in a simulated cross-silo setting under 20- and 40-round communication budgets.

Results show that static optimization achieves the strongest final predictive performance, driven by strong early-round behavior. In contrast, the agentic approach produces more gradual improvements across communication rounds and achieves competitive performance while reducing reliance on a separate hyperparameter search phase.

These findings demonstrate that hyperparameter strategy influences not only final outcomes but also the trajectory of learning itself. More broadly, this work highlights the importance of treating hyperparameter optimization as an integral component of federated training dynamics and suggests that agent-driven approaches provide a flexible and efficient alternative under communication constraints.

1. INTRODUCTION

Can the full potential of AI in healthcare be realized under current model training paradigms? From emergency triage to treatment planning, machine learning already shapes clinical decisions that affect patients every day. Realizing the promise of these data-driven approaches requires learning from data that span institutions and patient populations. However, in healthcare, clinical data are fragmented across organizations and governed by privacy, regulatory, and data ownership constraints that make large-scale centralized training impractical. As Xu et al. note, these constraints fundamentally limit the applicability of traditional centralized machine learning workflows in healthcare settings [1]. Rieke et al. argue that this limitation is especially consequential in healthcare and requires a fundamental rethinking of how models are trained across institutions, advancing federated learning as a necessary paradigm for scalable, data-driven care. As the authors state, they "envision a federated future for digital health" [2].

Federated learning coordinates model training across decentralized clients in settings where data cannot be centrally collected due to privacy, governance, or communication constraints [3]. As a result, model development proceeds using aggregated and delayed training signals rather than direct access to client data, breaking many assumptions that underlie traditional machine learning workflows. These limitations are further compounded by differences in how data are distributed across clients, as models must learn from locally skewed datasets rather than a single, unified population [4]. Li et al. show that this statistical heterogeneity can lead to unstable convergence and heightened sensitivity to optimization choices. These effects are difficult to diagnose in federated settings and may degrade performance or fairness without being immediately visible.

Beyond data heterogeneity, federated learning also changes how models can be evaluated and debugged during training. In centralized machine learning, model development relies on exploratory data analysis, direct inspection of training data, and error analysis to understand model behavior. These approaches assume full visibility into the data and the training process, which do not hold in federated settings. As Wang et al. note, federated learning limits access to client data and local training behavior, thereby forcing evaluation to rely on aggregated, privacy-preserving metrics instead [5]. This limited observability means that differences in model performance across clients can be obscured, even when overall metrics appear stable, making it difficult to determine whether changes in training behavior are due to data differences, optimization choices, or system effects. Without direct feedback, model evaluation becomes less informative, making optimization decisions more difficult to reason about and adjust during training.

Recent work has begun to explicitly study hyperparameter optimization in federated learning, acknowledging that hyperparameter choices can strongly influence convergence and final model quality yet are difficult to tune under federated constraints. Zhou et al. frame federated HPO as fundamentally constrained by limited access to global validation data and prohibitively high communication cost, motivating *single-shot* approaches that aim to select a single reasonable configuration before federated training begins [6]. While this reduces overhead, it also shifts tuning into a one-time decision made without feedback during training. Complementary research by Parra-Ullauri et al. demonstrates that existing HPO tools such as Optuna can be integrated with federated frameworks like Flower; however, each candidate configuration still requires many federated communication rounds to evaluate and produces only delayed, aggregated metrics [7]. As a result, federated HPO remains a slow and resource-intensive process that largely treats tuning as a fixed procedure rather than an adaptive process. Current approaches focus on making experimentation cheaper, but offer limited support for reasoning with partial feedback or for adjusting decisions as training progresses, leaving an important gap for methods capable of adaptively guiding optimization under limited observability.

This work explores hyperparameter tuning as a decision-making problem under limited observability, rather than as a one-time fixed optimization task. In federated settings, tuning decisions must be made using delayed, aggregated signals and without direct access to client data, making manual or static approaches difficult to reason about and slow to adapt. These characteristics suggest that hyperparameter optimization is better understood as a sequential process that unfolds over training, where decisions must adapt to partial and imperfect feedback.

Building on these limitations, recent work has explored agentic approaches for federated hyperparameter optimization. For example, Zhou et al. propose an agentic framework that adapts hyperparameters online during training using only compact, privacy-preserving summaries, demonstrating that agent-driven tuning can be feasible under federated constraints [6]. Similarly, work published on OpenReview explores the use of autonomous agents to guide hyperparameter adaptation in personalized federated learning, framing tuning explicitly as an ongoing reasoning process rather than a static search [8]. However, existing studies primarily emphasize performance gains or system design, leaving open questions about when agentic approaches are appropriate, what signals they rely on, and how they behave under realistic federated constraints.

This study evaluates agent-driven hyperparameter optimization in a controlled federated learning setting using the eICU Collaborative Research Database (eICU-CRD). The goal is not to establish optimal configurations, but to examine whether agentic approaches provide a suitable framework for guiding hyperparameter decisions under federated constraints. In particular, we compare baseline, static, and agentic strategies to assess their impact on both convergence behavior across communication rounds and final predictive performance. By analyzing how these strategies influence learning dynamics over time, this work aims to better understand the role of adaptive decision-making in federated model optimization, particularly in healthcare settings where data are distributed and communication is constrained.

2. BACKGROUND

2.1 Federated Learning in Healthcare

Federated learning (FL) was introduced by McMahan et al. [9] as a decentralized training paradigm in which multiple clients collaboratively train a shared model without transferring raw data to a central server. The Federated Averaging (FedAvg) algorithm implements this framework by allowing each client to perform local stochastic gradient descent updates on its private dataset before periodically aggregating model parameters at a coordinating server across communication rounds [9]. This iterative process enables joint model optimization across distributed data holders while preserving local data custody.

These structural properties make federated learning particularly relevant in domains where datasets are institutionally distributed rather than centrally curated. In healthcare research, clinical datasets are typically maintained by independent hospitals and health systems. Federated learning therefore provides a mechanism for joint model development across institutions without requiring the centralization of patient records. In practice, healthcare federated learning efforts typically involve collaborations among a limited number of stable institutional participants, each maintaining substantial local datasets. Within this cross-silo configuration, prior work shows that decentralized training supports medical imaging analysis, electronic health record modeling, and clinical outcome prediction across institutions [2].

2.2 Hyperparameter Optimization in FL

Hyperparameter optimization (HPO) is the process of selecting configuration parameters that govern model training but are not directly learned from data, such as the learning rate, regularization strength, or the number of local training epochs. In centralized machine learning workflows, hyperparameters are typically selected by training multiple candidate models and evaluating them on a validation dataset. Because the full training data and model behavior are directly observable, different configurations can be compared straightforwardly and iteratively.

In federated learning, hyperparameter choices play an equally important role in determining convergence behavior and final model quality. The setting in which those choices are made, however, is fundamentally different. Training occurs across multiple clients with distinct data distributions and dataset sizes. Prior work shows that many federated systems apply a single global set of hyperparameters across all clients, even when the underlying data characteristics differ substantially [10]. At the same time, Zawad et al. demonstrate that data quantity and distribution can meaningfully affect which hyperparameters perform best [10]. This creates a structural tradeoff: a single global configuration simplifies coordination, but may not reflect differences in client data.

One possible response is to tune hyperparameters separately for individual clients. However, this approach introduces additional cost. As the number of participating clients grows, the effort required to tune and evaluate separate configurations increases accordingly [10]. Moreover, federated learning restricts direct access to client data and local training dynamics. This limited observability means that hyperparameter decisions must rely on aggregated metrics reported to the server rather than direct inspection of client-level data or training dynamics. Compared to centralized workflows, tuning in federated learning is both more resource-intensive and less transparent.

Recent research has proposed methods to make hyperparameter tuning more practical in federated settings. However, federated HPO remains constrained by communication cost, heterogeneous client data, and limited visibility into local training behavior. These structural constraints shape how and when hyperparameter decisions can be made during training.

2.3 Static vs. Dynamic HPO

These structural differences have led to distinct approaches to hyperparameter optimization in federated learning. Broadly, existing methods fall into two categories: static strategies, in which hyperparameters are selected prior to training and remain fixed, and dynamic strategies, in which hyperparameters are adjusted during federated training. This distinction reflects differing assumptions about when tuning decisions should be made under federated constraints.

In static approaches, hyperparameters are selected prior to full federated training and remain fixed throughout all communication rounds. Zhou et al. propose FLoRA, which identifies a suitable configuration before federated training begins in order to reduce communication overhead [6]. Similarly, Parra-Ullauri et al. demonstrate that tools such as Optuna can be integrated with Flower to evaluate candidate configurations in federated environments [7]. In these settings, once a configuration is chosen, it remains unchanged during training. Static strategies

simplify coordination and system design, but they do not adapt to changes in optimization behavior observed across communication rounds.

Dynamic approaches instead treat hyperparameter selection as part of the training process itself. Rather than fixing parameters at the outset, dynamic methods adjust learning rates, local training epochs, or related settings during federated training based on observed model performance. In federated settings, such adjustments must rely on aggregated evaluation metrics reported by clients, since direct access to local data and training dynamics is restricted. Dynamic HPO therefore reframes tuning as an iterative process that unfolds over communication rounds rather than as a single pre-training decision.

2.4 Agentic HPO (LLM-driven)

Agentic hyperparameter optimization represents a specific form of dynamic HPO in which hyperparameter selection is delegated to an autonomous decision-making agent operating during training. Whereas dynamic methods broadly adjust hyperparameters over time, agentic approaches introduce an explicit controller that proposes updated configurations at each communication round based on observed performance.

In this formulation, hyperparameter tuning is treated as a sequential decision process embedded within federated optimization. After each communication round, the server aggregates evaluation metrics from participating clients. The agent receives these aggregated metrics and proposes updated hyperparameter values for subsequent rounds. Because federated learning restricts direct access to client data and local training dynamics, the agent must rely exclusively on these aggregated signals.

Recent work, including a concurrent OpenReview preprint by Abdullah Al Asif et al. [8], demonstrates that LLM-driven agentic controllers can be integrated into federated learning frameworks and can improve predictive performance relative to static baselines in benchmark settings. These results establish that round-by-round adaptive control is feasible under federated constraints.

By treating hyperparameter selection as a feedback-driven decision process, agentic HPO extends dynamic tuning beyond fixed schedules or heuristic adjustments. This formulation provides a structured way to adapt training behavior over communication rounds and motivates the evaluation conducted in this study.

2.5 Research Gap

Prior work shows that federated learning introduces constraints such as limited observability and communication cost [3], while heterogeneous client data further complicates optimization and increases sensitivity to training configuration [4,10]. Existing methods typically address these challenges through static optimization, which selects a single configuration prior to training, or through manual tuning based on aggregated metrics.

However, both strategies have clear limitations. Manual tuning is constrained by limited visibility into client-level training behavior, while static methods assume that a configuration chosen early will remain effective throughout training. In practice, configurations that perform well early or for one subset of clients may degrade as training progresses or as different client distributions increasingly influence the global model. As discussed in Section 2.3, static approaches simplify coordination but do not adapt to changes in optimization dynamics across communication rounds. These limitations are particularly relevant in healthcare, where data are distributed across institutions under privacy and governance constraints, limiting the use of centralized workflows [1].

Recent work has explored dynamic and agent-based methods that adapt hyperparameters during training, demonstrating that round-by-round adjustment is feasible under federated constraints [8]. However, these studies primarily focus on system design or performance improvements, and do not clearly evaluate whether adaptive hyperparameter control provides a meaningful advantage over static strategies in realistic federated settings.

This leaves an important gap: it remains unclear whether hyperparameter optimization in federated learning is better treated as a fixed configuration problem or as an adaptive decision process that evolves with training dynamics under limited observability.

This study addresses this gap by comparing baseline, static, and agentic hyperparameter strategies in a federated healthcare prediction task, examining both convergence behavior across communication rounds and final predictive performance.

3. STUDY DESIGN

This section describes the experimental design, including the dataset, federated configuration, partitioning strategy, and hyperparameter conditions evaluated. The goal is not to claim universal superiority or broad generalization across datasets or model classes, but to analyze convergence behavior under controlled federated conditions with limited observability and heterogeneous data.

3.1 Research Questions

The objective of this study is to evaluate how different hyperparameter optimization strategies affect federated training dynamics and predictive performance in a healthcare-motivated prediction task.

We consider the following research questions:

RQ$_1$: How does agentic hyperparameter optimization influence convergence behavior across communication rounds relative to baseline and static HPO?

RQ$_2$: How does agentic hyperparameter optimization affect final predictive performance relative to baseline and static HPO?

These questions examine not only final predictive performance but also how model behavior evolves across communication rounds, as training dynamics are central to optimization under decentralized constraints.

3.2 Prediction Task and Dataset

The prediction task is a binary classification problem: predicting whether an ICU stay is prolonged. A stay is defined as prolonged if the length of stay is three or more days (prolonged_stay = 1 if LOS ≥ 3 days, 0 otherwise). Prolonged ICU stays are clinically and operationally significant, as early identification of high-risk patients can support resource allocation and care planning. This task has been previously evaluated in both centralized and federated settings using the same dataset and preprocessing pipeline. The present study isolates the effect of hyperparameter optimization within the federated setting while keeping the task definition and data preparation constant.

Experiments were conducted using the eICU Collaborative Research Database (eICU-CRD v2.0) [11], a publicly available

multi-center ICU database containing over 200,000 patient unit stays from 208 hospitals across the United States between 2014 and 2015. The dataset contains de-identified demographic, physiologic, laboratory, and treatment data collected during routine critical care. Each instance corresponds to a single ICU stay identified by patientunitstayid.

Features were engineered by aggregating demographic variables, APACHE physiology variables, early vital signs, laboratory summaries, infusion data, respiratory features, and hospital characteristics. To support clinically meaningful early prediction, features were derived from information available within the first 24 hours of ICU admission. Feature construction was implemented through a series of SQL views that merge and aggregate relevant tables into a consolidated modeling dataset. These views were version-controlled to ensure consistency and traceability of feature construction across experiments. The resulting dataset exhibits non-independent and identically distributed (non-IID) data across clients, reflecting the natural heterogeneity of real-world clinical data.

After filtering corrupted records and consolidating engineered feature views, the final modeling dataset contains 199,646 ICU stays. The class distribution is moderately imbalanced, with approximately 25% prolonged stays and 75% non-prolonged stays.

The dataset exhibits substantial missingness due to irregular sampling of clinical variables. Numeric features were handled using simple imputation, while categorical features included standardized "unknown" categories where appropriate. For selected variables, binary missingness indicators were incorporated to preserve signal associated with missing measurements.

All preprocessing steps, including imputation, categorical encoding, feature scaling, and annotation-based category standardization, were implemented within a scikit-learn pipeline. Raw categorical values were mapped to predefined canonical categories using an annotation configuration, thereby reducing encoding variability and ensuring consistency in the resulting feature space.

3.3 Federated Partitioning Strategy

The consolidated modeling dataset is partitioned into three clients based on geographic hospital region. The clients are defined as follows:

- **Client Midwest:** hospitals in the Midwest region
- **Client South:** hospitals in the South region
- **Client Other:** hospitals in the West, Northeast, and records with unspecified region (treated as "Other")

To characterize the resulting heterogeneity across clients, Table 1 reports sample counts and class distributions for each partition.

Table 1. Client-level data distribution.

Client	Samples	% Positive (LOS ≥ 3 days)
Midwest	65,700	25.5%
South	59,980	27.5%
Other	73,966	21.9%
Total	199,646	24.8%

Differences in class prevalence across regions introduce moderate heterogeneity, reflecting realistic cross-institutional variation in healthcare data. The partitioning preserves natural regional variation while maintaining a roughly balanced distribution of samples across clients, enabling controlled comparison with a manageable number of participants. No overlap exists between client partitions at the patient or record level, ensuring independence across clients.

Each client loads its regional subset from the consolidated feature view. Data splitting is performed prior to partitioning using a global 60/20/20 train, validation, and test split applied consistently across all clients. During federated training, clients operate on their respective portions of the training split, while validation metrics are computed locally and aggregated at the server after each communication round.

Although the experiment is executed on a single machine, clients operate as logically independent participants within the federated training process.

3.4 Model and Federated Configuration

Federated training is implemented using the Flower framework [12] with the Federated Averaging (FedAvg) algorithm in a simulated cross-silo configuration, where logically distinct clients operate on partitioned data within a shared runtime environment. All clients participate in every communication round, and the server aggregates client updates after each round.

Each client trains the same model class, namely a scikit-learn SGDClassifier configured with logistic loss (log_loss). This model was selected because it supports iterative training across communication rounds using warm-started optimization, making it compatible with round-based federated updates where model parameters are incrementally refined over time. In our implementation, the number of client-side local epochs per round is controlled via the classifier's max_iter parameter, which is updated from the current hyperparameter configuration at each communication round.

While neural network architectures are common in modern federated learning, a linear model was chosen in this study to provide a controlled setting for analyzing optimization dynamics, allowing the effects of hyperparameter strategies on convergence behavior to be isolated without additional complexity introduced by deep architectures.

All clients apply the same pre-fitted preprocessing pipeline, which is stored as a serialized scikit-learn artifact and loaded locally prior to model training. The pipeline defines a fixed feature transformation sequence and column order, which is validated at load time. This ensures that each client operates on an identical feature space and produces compatible model updates.

Model exchange between clients and server occurs strictly through model parameters. At the end of each communication round, clients return model weights and intercepts, which are aggregated at the server using standard FedAvg with sample-size weighting across clients. Consistent parameter alignment is ensured through standardized model initialization and preprocessing. The updated global parameters are then broadcast to clients for the next round. No raw data are exchanged, preserving data locality and aligning with privacy constraints typical of healthcare federated learning environments.

Across all experimental conditions, the following components are held constant:

- Federated implementation: Flower with FedAvg
- Model architecture: SGDClassifier with logistic loss

- Shared preprocessing pipeline and feature schema
- Client composition and full participation each round
- Total number of communication rounds
- Random seed (42)

Holding these factors fixed ensures that observed differences can be attributed to the hyperparameter strategy. Model initialization is standardized across clients using a shared set of class labels and initial parameter shapes, ensuring that parameter aggregation is well-defined from the first communication round.

Figure 1 illustrates the overall federated training architecture and the interaction between clients, the server, and the hyperparameter controller.

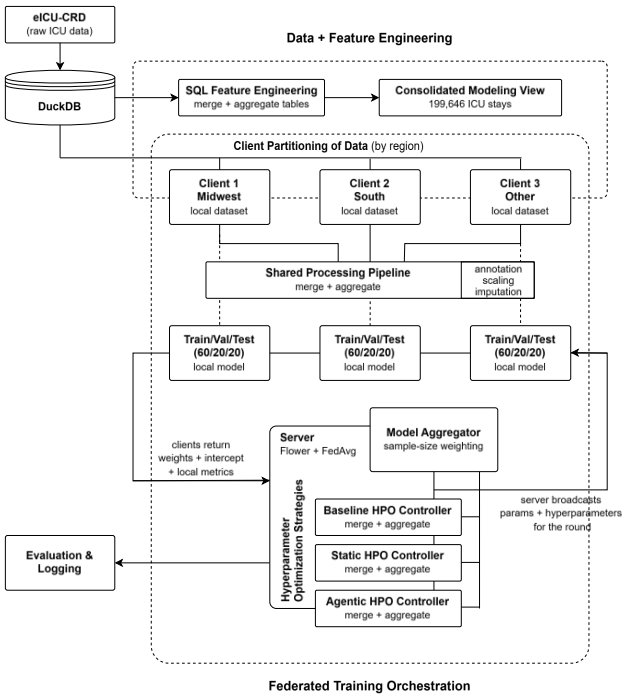

Figure 1. Approach overview.

3.5 Hyperparameter Strategies

Three hyperparameter strategies are considered: a fixed baseline, a statically tuned configuration, and an adaptive agentic approach. All strategies operate within the same federated configuration described in Section 3.3 and differ only in how hyperparameters are selected and updated across communication rounds.

3.5.1 Baseline

The baseline condition uses a fixed set of manually selected hyperparameters chosen based on reasonable defaults and prior experimentation. These values remain constant for all clients and across all communication rounds. This configuration provides a reference point for assessing the impact of automated tuning strategies.

3.5.2 Static HPO

Static hyperparameter optimization selects a single global configuration prior to full federated training. The configuration is determined using Optuna [13], a general-purpose hyperparameter optimization framework integrated with the federated training loop. Candidate configurations are evaluated under federated training constraints, and the configuration with the highest aggregated validation ROC-AUC across clients, weighted by evaluation sample size, is selected.

To control tuning cost, each Optuna trial is evaluated over 5 communication rounds rather than the full training budget. After selection, the chosen configuration is trained for the same full communication budget used in the final experimental comparison, ensuring a consistent evaluation protocol across all strategies. The search space matches that of the agentic condition so that both approaches operate over identical bounded hyperparameter ranges. Once selected, the configuration remains fixed for the duration of federated training. This separation between hyperparameter search and final evaluation ensures that convergence behavior reflects the selected configuration rather than ongoing adaptation during evaluation.

3.5.3 Agentic HPO

Agentic hyperparameter optimization embeds hyperparameter selection within the federated training loop using an LLM-based controller implemented via the OpenAI Agent SDK. The controller operates at the server level and proposes updated hyperparameters after each communication round.

At the end of each round, the server aggregates evaluation metrics, including ROC-AUC and log loss, across participating clients. These aggregated metrics, together with a history of prior configurations and outcomes, are provided to the controller as a state representation capturing recent metric trends, summary statistics, and bounded context. The controller does not observe raw client data or local training dynamics, and instead operates solely on aggregated metrics and derived state information. Based on this information, the controller proposes a hyperparameter configuration for the subsequent round. This process is repeated each round, forming a closed-loop feedback system linking observed performance to hyperparameter selection.

The controller operates over the same bounded hyperparameter search space defined for the static HPO condition. The hyperparameters subject to adaptation are:

- Number of local training epochs
- Regularization penalty type (l1, l2, elasticnet)
- Learning rate schedule (optimal, constant, adaptive)
- Initial SGD step size (eta0), subject to schedule-dependent constraints

Class weighting is currently held fixed and is not subject to adaptation.

All proposed values are validated against predefined ranges before being applied. If validation fails or the controller invocation is unsuccessful, the system reuses the previous configuration to maintain training continuity.

The controller operates with a low-temperature setting (0.2), a fixed-length history window, and a global random seed (42) to reduce stochastic variability. Unlike approaches that rely on a single preselected configuration prior to training, this method enables hyperparameters to be adapted sequentially during training based on observed performance across rounds, rather than remaining fixed as in the static condition.

This sequential adaptation is enabled by a structured decision space defined by explicit constraints and schema-validated outputs, ensuring a controlled and reproducible optimization process. Full prompt structure and output schema are provided in

Appendix A, with implementation details and round-level decision logs available in the project repository to support reproducibility and independent verification.

3.5.4 Experimental Control

All three strategies are evaluated under the same communication budget, number of rounds, client participation schedule, preprocessing pipeline, model architecture, and bounded hyperparameter search space. This ensures that observed differences in convergence behavior or predictive performance can be attributed to the hyperparameter strategy itself rather than to differences in training setup.

3.6 Evaluation Metrics

Model performance is evaluated using ROC-AUC, log loss, and accuracy. ROC-AUC is emphasized for its robustness to class imbalance and its relevance to ranking-based clinical decision support, while log loss provides complementary information about probabilistic calibration. Accuracy is reported for interpretability but is not used for model selection.

Log loss is computed on each client using scikit-learn's log_loss function (mean loss per sample) and aggregated across clients using a weighted average based on the number of evaluation samples per client. The resulting values are larger than typical binary classification log loss due to probability saturation caused by extreme feature values in the eICU dataset, which can lead to overconfident predictions and poor probability calibration. For this reason, ROC-AUC is treated as the primary evaluation metric in this study. Absolute log loss values should therefore be interpreted with caution. Comparisons across methods and rounds provide the most meaningful insights.

Evaluation metrics are computed locally on each client's evaluation split at the end of each communication round and aggregated at the server. This enables both final-round performance comparison and analysis of convergence behavior.

For the static HPO condition, validation ROC-AUC is used to select the best-performing configuration. In the agentic condition, aggregated ROC-AUC and log loss are provided to the controller at each round as part of the decision context.

In addition to final predictive performance, convergence dynamics are analyzed by tracking metric trajectories over successive communication rounds, enabling assessment of stability, responsiveness to hyperparameter adjustments, and relative convergence speed across strategies.

4. EXPERIMENTS

4.1 Experimental Setup

Experiments were conducted under two communication budgets: 20 and 40 federated training rounds. The 20-round setting represents a limited communication budget, while the 40-round setting captures a longer training horizon, enabling comparison of both early-stage behavior and longer-term convergence. Together, these settings enable comparison of hyperparameter strategies in terms of both convergence behavior and final predictive performance.

For the static HPO condition, hyperparameters were selected using Optuna [13] as described in Section 3.5. A total of 15 candidate configurations were evaluated, and each trial was run for 5 communication rounds on the training split with evaluation on the validation split. The configuration with the best aggregated validation ROC-AUC was then selected and used for final comparison runs under both the 20-round and 40-round communication budgets. The model was trained on the combined train+validation data and evaluated on the test split. The search settings used for this phase were hpo-n-trials = 15 and hpo-num-rounds = 5.

The agentic HPO condition followed the configuration described in Section 3.5. In this setting, the agent operated at the server level and proposed updated hyperparameters after each communication round based on aggregated evaluation metrics and recent training history. For each communication budget, the agentic search phase was conducted on the training split and evaluated on the validation split. The best-performing hyperparameters were then selected and used in a separate final evaluation run. The best-performing configuration was selected using a composite score that combines ROC-AUC and log loss to balance discrimination and calibration, with the formulation provided in Appendix A.

The same training and evaluation protocol was used, with models trained on the combined train+validation data and evaluated on the test set. To isolate the quality of hyperparameter configurations identified by each method from the effects of continuous adaptation, the agentic approach is evaluated in a one-shot setting. In this setup, hyperparameters identified during the adaptive search phase are fixed for evaluation, providing a controlled "configuration quality" benchmark that enables direct comparison with static HPO.

However, while the agent proposes updates during the search phase, the reported results use fixed hyperparameters, thereby isolating configuration quality but not fully capturing the intended use of agentic control as a continuously adaptive process. As a result, convergence behavior reflects the properties of the selected configuration rather than ongoing adaptive control during evaluation.

All agentic runs used a temperature of 0.2, and all experiments were executed in the simulated cross-silo configuration described in Section 3.3, with three clients participating in every communication round. The federated algorithm (FedAvg), preprocessing pipeline, model architecture, client composition, and random seed (42) were held constant across all runs. This ensures that the comparison remains focused on the hyperparameter strategy rather than differences in data partitioning, initialization, or training.

Evaluation metrics were recorded at the end of each communication round and aggregated at the server for analysis. In addition to ROC-AUC, log loss is also reported to provide a complementary view of model behavior. However, as discussed in Section 3.6, absolute log loss values are influenced by probability saturation effects in the dataset and should be interpreted cautiously. Both convergence patterns and final-round performance are reported in the following sections.

4.2 Reproducibility and Artifacts

To support reproducibility and transparency, all experiments were implemented within a modular federated learning pipeline. The system includes components for data preprocessing, client training, server aggregation, and hyperparameter control, as described in Section 4.1.

Several artifacts were generated and maintained throughout the study to ensure consistency across experimental runs. These include a serialized scikit-learn preprocessing pipeline used to

enforce consistent feature transformations across clients, as well as model initialization metadata specifying feature dimensions, class labels, and parameter structure.

Hyperparameter configurations were preserved for both optimization strategies. For static HPO, Optuna trial results were recorded, including all evaluated configurations and their corresponding validation metrics. For the agentic approach, round-by-round hyperparameter proposals and associated performance metrics were logged to capture the adaptive decision process over time.

In addition, training logs were maintained for each communication round, including aggregated ROC-AUC, log loss, and accuracy across all strategies. Experiment-level configuration files were also retained, specifying communication budgets, client participation settings, and hyperparameter search parameters.

All experiments were executed using a fixed random seed (42) to ensure reproducibility. The separation between hyperparameter search and final evaluation was enforced programmatically to prevent data leakage.

To further support reproducibility, the full implementation of the experimental pipeline, including data preprocessing, federated training, and hyperparameter optimization components, is available in a public repository [14]. The repository contains all necessary code, configuration files, and documentation required to replicate the experiments described in this study. In addition, all experimental artifacts, including preprocessing pipelines, hyperparameter configurations, and training logs, were preserved to support reproducibility and future extension of this work.

4.3 Hyperparameter Trajectory Analysis

To better understand how the agent influences training dynamics, we examine how it adjusts hyperparameters over communication rounds. As shown in Table 2, the agent initially explores a wide range of configurations, including more local training epochs and varying regularization strategies. This early exploration phase corresponds to rapid improvements in validation AUC during the first five rounds, increasing from 0.554 to 0.640.

Table 2. Agentic Hyperparameter Trajectory (20 Rounds).

Round	Epochs	LR Schedule	Eta0	Penalty	AUC	Loss
1	5	optimal	0	l2	0.554	11.8
2	8	adaptive	0.003	elasticnet	0.621	8.65
3	3	adaptive	0.002	l2	0.625	8.48
4	8	adaptive	0.003	l2	0.633	8.7
5	3	adaptive	0.002	elasticnet	0.64	8.61
10	3	adaptive	0.002	l2	0.639	9.17
11	3	adaptive	0.002	l2	0.646	8.68
12	3	adaptive	0.002	l2	0.648	8.63
13	3	adaptive	0.001	l2	**0.65**	8.84
15	3	adaptive	0.001	l2	0.637	9.41
20	3	adaptive	0.001	l2	0.634	9.54

As training progresses, the agent reduces the magnitude of its adjustments and converges toward a stable configuration characterized by fewer local epochs, an adaptive learning rate schedule, and small learning rate values. In particular, the configuration stabilizes at 3 local epochs, an adaptive learning rate, and small eta0 values. From approximately round 11 onward, the agent enters an exploitation phase, making only minor refinements to the learning rate while maintaining a largely consistent configuration. In contrast, static HPO performs adaptation prior to training, whereas agentic HPO enables hyperparameter adjustment during training itself.

This transition from exploration to stabilization reflects how the agent refines hyperparameters during the tuning phase, ultimately identifying an effective hyperparameter configuration, which is evaluated in the next section through convergence over communication rounds.

For reproducibility, full round-by-round logs of the agentic tuning process are available in the project repository [14].

4.4 Convergence Dynamics

In federated learning, the global model evolves over sequential training rounds as client updates are iteratively aggregated. Hyperparameters such as learning rate and local training epochs influence the magnitude and direction of these updates, thereby shaping the trajectory of optimization. Although all configurations perform iterative refinement, differences in how hyperparameters are controlled may substantially affect convergence speed, stability, and plateau behavior. Because results are based on single-run experiments, emphasis is placed on consistent patterns across rounds and strategies rather than small absolute differences in final metric values. This section therefore focuses on round-by-round performance rather than solely on final predictive outcomes.

4.4.1 20-Round Budget

Figure 2 presents global validation ROC-AUC trajectories across 20 communication rounds. Table 3 summarizes convergence statistics, including AUC at Round 1, AUC at Round 20, total change in AUC (Δ AUC), and the average absolute change across the final five rounds (Last-5 Δ).

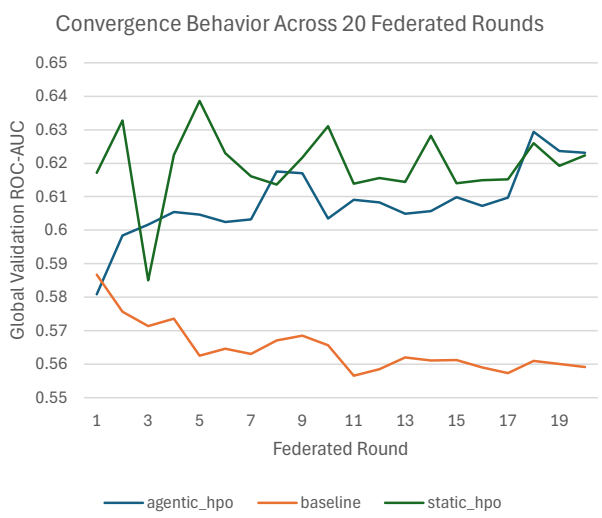

Figure 2. Validation ROC-AUC across 20 federated rounds.

Table 3. Convergence Summary (20 rounds).

Method	AUC R1	AUC R20	Δ AUC	Last-5 Δ
Agentic	0.58082	0.62312	0.0423	0.0158
Baseline	0.58676	0.55913	−0.02762	0.00015
Static	0.61718	0.62241	0.00523	0.00743

Across the 20-round budget, the three strategies exhibit distinct learning trajectories.

The statically tuned configuration begins at substantially higher validation performance relative to both the baseline and agentic conditions. This reflects the effect of pre-training hyperparameter selection, which provides a strong initial starting point. However, this early advantage is not strictly maintained. Performance declines in the initial rounds before recovering later in training, resulting in only modest net improvement from Round 1 to Round 20. The relatively small Last-5 Δ indicates that performance stabilizes toward the end of the training horizon.

The agentic configuration exhibits the largest overall increase in ROC-AUC from Round 1 to Round 20. Unlike the static condition, it does not begin with a strong initial advantage, but instead improves gradually over successive rounds. This reflects hyperparameters identified during the agentic search phase that produce a more incremental improvement pattern. However, round-to-round variation is more pronounced than in the baseline condition, indicating that the selected configuration produces a more variable optimization trajectory. Late-round changes remain moderate, indicating stable convergence behavior. This highlights how hyperparameters identified through agentic search shape how optimization unfolds over time, rather than relying on strong initial conditions.

The baseline configuration shows minimal overall improvement across rounds. While small fluctuations occur, total change from Round 1 to Round 20 is negative. The near-zero Last-5 Δ indicates early plateauing under fixed default hyperparameters, with little benefit from continued training within this budget.

Overall, under a 20-round communication budget, differences across strategies are primarily reflected in initial positioning and cumulative improvement. Static tuning provides a strong starting point but limited net gain, while agentic search produces the largest improvement over time through a more gradual upward trajectory. This suggests that static tuning primarily influences initial model quality, whereas agentic search more strongly affects how performance evolves across rounds.

4.4.2 40-Round Budget

Figure 3 extends the convergence analysis to 40 communication rounds. Table 4 reports corresponding convergence statistics.

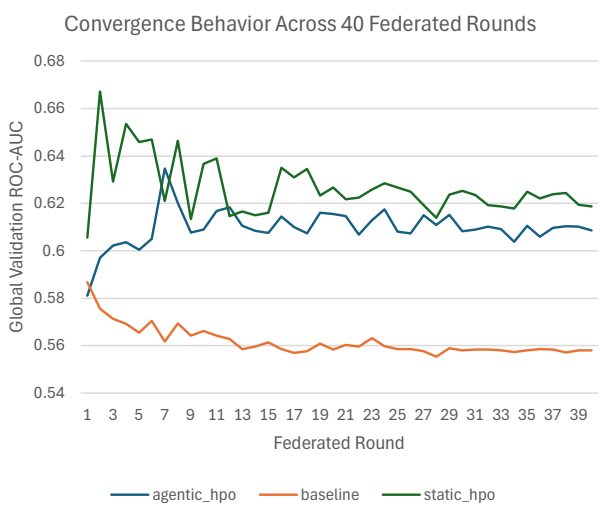

Figure 3. Validation ROC-AUC across 40 federated rounds.

Table 4. Convergence Summary (40 rounds).

Method	AUC R1	AUC R40	Δ AUC	Last-5 Δ
Agentic	0.58105	0.60864	0.02758	0.00274
Baseline	0.58676	0.55806	−0.02870	−0.00053
Static	0.60558	0.61876	0.01317	−0.00334

Under the extended communication budget, differences in convergence behavior become more apparent.

The statically tuned configuration again begins with a strong initial ROC-AUC and reaches its highest performance within the first few rounds. This early spike is followed by a decline and subsequent stabilization, with performance settling into a narrower range for the remainder of training. Although the final AUC remains higher than the other strategies, the majority of performance gains occur early rather than being accumulated over time. The negative Last-5 Δ reflects reduced fluctuation in later rounds, indicating convergence to a stable plateau after this initial adjustment period.

The agentic configuration shows a more gradual improvement pattern over the first several rounds, reaching peak performance in the mid-training phase. After this point, performance stabilizes and slightly declines toward the end of training. While the overall change from Round 1 to Round 40 remains positive, improvements are not sustained throughout the entire training process. This suggests that the hyperparameters identified during the agentic search phase support early gains but do not consistently translate into improved final performance, highlighting a distinction between optimization trajectory and final outcomes.

The baseline configuration remains largely unchanged across rounds, with a slight downward trend and minimal variability. Performance plateaus early and does not benefit from additional communication rounds, reinforcing the limitations of fixed default hyperparameters in this setting.

Overall, extending training from 20 to 40 rounds does not uniformly improve performance across strategies. Static tuning concentrates gains early in training, while agentic search supports more gradual improvements that stabilize over time. The baseline configuration remains largely unaffected by increased communication, indicating that additional rounds alone are insufficient without effective hyperparameter selection.

4.5 Final Predictive Performance

To address RQ2, we compare final-round predictive performance across baseline, static HPO, and agentic HPO strategies under both communication budgets. Performance is evaluated using aggregated validation ROC-AUC and log loss, with accuracy reported for reference. These results should be interpreted in the context of a distinction between discrimination and calibration performance, where improvements in ROC-AUC do not necessarily correspond to improvements in probabilistic calibration.

4.5.1 20-Round Results

Table 5 summarizes final-round metrics after 20 communication rounds. Under this limited communication budget, final performance reflects early-stage optimization behavior rather than full convergence.

Table 5. Final Predictive Performance (20 rounds).

Method	Final AUC	Final Loss	Final Accuracy
Agentic	0.6231	8.8182	0.7248
Baseline	0.5591	9.4279	0.7381
Static	0.6224	7.1652	0.739

Under the 20-round budget, the statically tuned and agentic configurations achieve nearly identical final ROC-AUC, both clearly outperforming the baseline. While the difference between static and agentic is minimal in terms of AUC, static tuning yields a noticeably lower final log loss, suggesting relatively improved probability calibration compared to the other strategies, although absolute values should be interpreted cautiously due to the calibration effects described in Section 3.6. This further illustrates the distinction between discrimination and calibration performance, where improvements in ROC-AUC do not necessarily correspond to improvements in probabilistic calibration.

The agentic configuration improves meaningfully over the baseline in both ROC-AUC and log loss, but does not clearly surpass the statically tuned configuration. However, in contrast to the static configuration's early fluctuations observed in Section 4.3, the agentic approach demonstrates a more consistent improvement pattern across rounds, suggesting a more stable learning trajectory under constrained communication. Accuracy values remain similar across methods and do not materially change the relative ordering.

Overall, under a limited communication budget, both static and agentic hyperparameter strategies improve predictive performance over the baseline. Static tuning provides slightly stronger final calibration, while agentic search achieves comparable discrimination performance.

4.5.2 40-Round Results

Table 6 presents final performance metrics after 40 communication rounds.

Table 6. Final Predictive Performance (40 rounds).

Method	Final AUC	Final Loss	Final Accuracy
Agentic	0.6086	10.9035	0.689
Baseline	0.5581	9.7102	0.7304
Static	0.6188	4.582	0.715

Under the extended communication budget, the statically tuned configuration achieves the highest final ROC-AUC and the lowest log loss among all strategies. The improvement in log loss is particularly pronounced, suggesting relatively better probability calibration compared to both agentic and baseline conditions, although absolute values should be interpreted cautiously due to the calibration effects described in Section 3.6.

The agentic configuration continues to outperform the baseline in ROC-AUC but does not match the performance of the statically tuned configuration. Higher log loss suggests relatively weaker calibration, though this should be interpreted in the context of the calibration effects described in Section 3.6. While agentic search supports gradual improvements in AUC, these gains do not translate into superior final performance in this setting, highlighting a distinction between optimization trajectory and final outcomes.

The baseline configuration remains largely unchanged, with consistently lower ROC-AUC and higher loss compared to the tuned approaches.

Overall, under a 40-round communication budget, static hyperparameter optimization provides the strongest final predictive performance across both discrimination and calibration metrics. Agentic search improves upon baseline performance but does not surpass the statically tuned configuration. This result is consistent with the convergence behavior observed in Section 4.3, where static tuning achieves strong early performance that carries through to the final rounds.

4.6 Comparative Analysis

Taken together, these results show that hyperparameter strategy influences both convergence dynamics and final predictive performance, with effects that depend on the available communication budget.

Under the 20-round budget, static hyperparameter optimization provides the strongest final predictive performance. As shown in Section 4.3, static tuning achieves favorable early positioning and reaches a stable plateau quickly. This early advantage carries through to the final round, resulting in slightly higher ROC-AUC and substantially lower log loss. In contrast, the agentic configuration exhibits the largest cumulative improvement across rounds, despite starting from a weaker initial position. This suggests that while static tuning benefits from strong initialization, agentic search more actively influences how learning develops over time. In this setting, pre-training search is sufficient to achieve strong final performance, but agentic control demonstrates a more progressive convergence pattern.

Under the 40-round budget, differences in convergence behavior become more apparent. Static tuning maintains strong early performance and ultimately achieves the highest final ROC-AUC and lowest log loss. However, as observed in Section 4.3, this performance is largely established in the early rounds and is not consistently improved through continued training. The agentic configuration, by contrast, shows a more gradual improvement pattern across rounds, reflecting a trajectory that continues to evolve beyond the initial stages of training. During the agentic search phase, hyperparameter proposals evolved gradually in response to observed performance. This indicates that the agent's decisions were guided by meaningful feedback rather than chance. Although this does not result in higher final predictive performance, it highlights a key distinction: agentic search shapes how optimization progresses across communication rounds, rather than concentrating gains in early initialization. This more gradual improvement pattern suggests greater sensitivity to round-by-round optimization dynamics. This indicates that agentic hyperparameter optimization acts not only as a selection mechanism, but as a process that shapes how learning unfolds over time.

Another important difference between the two approaches lies in how hyperparameter search cost is incurred. Static HPO evaluates multiple candidate configurations through a separate trial-based search phase, whereas agentic HPO integrates hyperparameter adaptation directly into the federated training process. As a result, the agentic approach achieves performance comparable to static tuning while reducing reliance on a separate communication-intensive search phase, providing a more efficient alternative in settings where communication rounds are costly.

The baseline configuration remains relatively stable across both budgets, with consistently lower ROC-AUC and higher loss. This indicates that the default hyperparameters provide stable but limited performance and do not substantially benefit from additional communication rounds.

Overall, these findings indicate that the impact of hyperparameter strategy is not uniform across training lengths. Static optimization provides a strong initial configuration that carries through to final performance, particularly when early gains dominate the training process. In contrast, agentic control produces a more progressive improvement pattern across rounds, suggesting a greater capacity to shape optimization dynamics over time, even when this does not result in higher final performance within the evaluated budgets.

In federated environments characterized by communication constraints and delayed feedback, this distinction is important. The choice between static and adaptive hyperparameter strategies should therefore be considered in relation to the expected communication budget and training length, rather than evaluated in isolation. These findings highlight the importance of evaluating hyperparameter strategies across multiple training regimes, as conclusions drawn under limited communication budgets may not generalize to longer training horizons.

4.7 Summary of Findings

Across both communication budgets, hyperparameter strategy meaningfully shapes both convergence dynamics and final predictive performance. Static hyperparameter optimization consistently achieves the strongest final performance, driven by strong early-round behavior that largely determines outcomes under both 20-round and 40-round training budgets.

Agentic hyperparameter control, by contrast, produces a more gradual and progressive improvement pattern across communication rounds. While this does not translate into superior final predictive performance under the evaluated conditions, it highlights a key distinction: agentic search influences how performance evolves over time, rather than concentrating gains in early initialization.

Importantly, agentic HPO achieves performance comparable to static tuning while reducing reliance on a separate communication-intensive search phase. By integrating hyperparameter adaptation directly into federated training, the agentic approach provides a more efficient alternative in settings where communication rounds are costly.

Taken together, these findings indicate that the effectiveness of hyperparameter strategy depends on both communication budget and training length. Static tuning is well-suited to settings where early performance is critical, while agentic control offers a more dynamic approach that shapes learning behavior across rounds.

While this study demonstrates the feasibility and impact of agent-driven hyperparameter adaptation in a controlled cross-silo setting, further investigation is needed to evaluate its behavior under larger client populations, heterogeneous data distributions, and varying communication regimes.

5. THREATS TO VALIDITY

Several factors may influence the interpretation of these results, particularly with respect to experimental design, measurement, and generalizability. Care was taken throughout the study to control for these factors and isolate the effect of hyperparameter strategy. These considerations are discussed in terms of internal, construct, and external validity.

5.1 Internal Validity

To ensure a fair comparison, all configurations were evaluated using the same dataset, preprocessing pipeline, model architecture, client composition, and training procedure. The only variable that differed across conditions was the hyperparameter strategy itself. Hyperparameter search and final evaluation were explicitly separated, with configurations selected using a validation split and the test set reserved exclusively for final evaluation. This protocol was adopted to eliminate earlier sources of leakage and ensure strict separation between tuning and testing. A fixed random seed was used across all runs to reduce stochastic variation and improve reproducibility.

Differences in hyperparameter search procedures introduce an additional source of variation across conditions. Static HPO evaluates multiple candidate configurations prior to final training, while agentic HPO adapts parameters during training. To improve comparability, the agentic approach was evaluated in a one-shot setting, where hyperparameters identified during the search phase were fixed for a separate final evaluation run. This aligns the evaluation process more closely with static optimization and helps ensure that observed differences are attributable to the strategies themselves.

All reported results are based on single-run experiments with a fixed random seed. As a result, observed differences in performance, particularly small differences in ROC-AUC, should not be interpreted as statistically significant at the level of individual point estimates. The findings are intended to characterize qualitative differences in convergence behavior and optimization dynamics rather than to establish definitive performance superiority between methods. Future work will incorporate repeated runs and statistical analysis to quantify variability and assess significance.

5.2 Construct Validity

Measurement-related threats were addressed by evaluating performance using both ROC-AUC and log loss at each communication round. These metrics provide complementary perspectives on model behavior, capturing both ranking performance and probabilistic calibration. Evaluation was performed consistently across rounds using a fixed validation and test protocol. While these metrics capture key aspects of predictive performance, they may not fully reflect all dimensions of clinical utility.

5.3 External Validity

The experimental setup reflects a realistic cross-silo federated learning scenario using real-world clinical data from the eICU Collaborative Research Database. The dataset includes diverse patient populations, hospital characteristics, and clinically meaningful features derived from early ICU stay data. Features were constructed from information available within the first 24 hours of ICU admission, reflecting a clinically relevant prediction setting in which early information is used to anticipate a prolonged length of stay.

At the same time, the study was conducted with a fixed number of clients in a controlled environment. While this enables clear, consistent comparisons of hyperparameter strategies, further evaluation across larger client populations and more

heterogeneous data distributions would help extend these findings to a broader range of federated settings.

6. RELATED WORK

Hyperparameter optimization in federated learning introduces additional complexity compared to centralized machine learning due to the need for repeated distributed training. Evaluating candidate configurations typically requires multiple federated runs, each incurring coordination and communication overhead, making standard search approaches expensive in federated settings.

Prior work has explored integrating established optimization frameworks into federated learning systems. Parra-Ullauri et al. [7] demonstrate the use of Optuna within the Flower framework to systematically evaluate hyperparameter configurations across distributed clients. While this approach can improve model performance, it relies on repeated training trials, increasing the computational and communication cost associated with tuning.

To address this limitation, Zhou et al. [6] propose FLoRA, a single-shot hyperparameter optimization approach for federated learning. In this method, candidate configurations are evaluated prior to full training, after which a single configuration is selected for subsequent use. This reduces the need for repeated federated runs and lowers communication overhead. However, because hyperparameters are fixed after selection, the approach assumes that a single configuration remains effective throughout all communication rounds, without adapting to changes during training.

Building on these approaches, recent work has introduced adaptive methods for hyperparameter optimization. Abdullah Al Asif et al. [8] propose an agent-based method in which hyperparameters are updated during training based on observed performance. Their results show that this approach can improve predictive performance while reducing the need for multiple full training runs. This work focuses primarily on performance improvements and system design. It does not explicitly analyze convergence behavior across communication rounds or compare adaptive strategies to static approaches under different communication budgets.

This study builds on these prior approaches by comparing static and adaptive hyperparameter strategies within a consistent experimental setting, with a focus on convergence behavior across communication rounds in addition to final predictive performance. By evaluating these strategies under different communication budgets, this work provides additional insight into their impact on training behavior in federated learning.

7. CONCLUSION

This study reframes hyperparameter optimization in federated learning as a sequential decision-making problem under limited observability, shifting the focus from static configuration selection to adaptive control of training dynamics. Instead of focusing solely on final predictive performance, this work analyzes how different approaches influence learning over time.

The results show that both static and agent-driven hyperparameter strategies provide meaningful advantages. Static hyperparameter optimization yields strong early performance by identifying an effective configuration before training, leading to faster convergence under limited communication budgets. In contrast, the agentic approach supports gradual, iterative adaptation, allowing hyperparameters to evolve in response to observed performance. Although this does not consistently result in higher final predictive performance, it reveals a fundamentally different mode of optimization in which learning is shaped continuously rather than determined by a fixed initial configuration.

A central finding of this work is that hyperparameter strategy shapes not only final predictive performance but also the trajectory of learning itself. Static tuning concentrates gains early in training, while the agentic approach distributes improvements more gradually across communication rounds. This distinction is especially important in federated healthcare settings, where feedback is delayed and aggregated, and optimization decisions must be made without direct visibility into client-level behavior.

From a systems perspective, efficiency is a central consideration. Static HPO requires a separate trial-based search phase, increasing communication and computational cost. In contrast, the agentic approach enables hyperparameter adaptation to be performed as part of federated training itself, reducing reliance on a separate communication-intensive search process. This integration allows effective hyperparameter configurations to be identified within a single adaptive training run while maintaining comparable performance. As a result, agent-driven hyperparameter control offers a more practical and efficient alternative in federated healthcare environments where communication is constrained.

Taken together, these findings suggest that hyperparameter strategy is not merely a preprocessing step, but an integral component of federated training design. In healthcare applications, agent-driven approaches provide a flexible way to guide optimization over time within the constraints of decentralized data.

Future work can extend this approach by expanding the set of tunable hyperparameters to include factors such as class weighting and regularization strength, which may improve model calibration and robustness in clinical prediction tasks. In addition, incorporating convergence-aware mechanisms at the federated level, such as detecting stabilization in validation performance across communication rounds and terminating training early, may further improve efficiency by avoiding unnecessary rounds once performance has plateaued. These directions suggest that combining agent-driven hyperparameter control with convergence-aware training can enable more efficient and adaptive federated learning systems, particularly in healthcare environments where communication and data access are inherently constrained.

More broadly, this work shows that treating hyperparameter optimization as a sequential, feedback-driven process provides a principled framework for improving federated learning under limited observability and costly communication. In such settings, adaptive approaches offer a practical alternative to static, pre-training optimization strategies, particularly in federated healthcare environments. In this way, agent-driven hyperparameter optimization offers a practical path toward more adaptive and efficient federated learning systems.

8. REFERENCES

[1] Xu, J., Glicksberg, B. S., Su, C., Walker, P., Bian, J., and Wang, F. 2021. Federated learning for healthcare informatics. *Journal of Healthcare Informatics Research* 5, 1 (Mar. 2021), 1–19. https://doi.org/10.1007/s41666-020-00082-4.

[2] Rieke, N., et al. 2020. The future of digital health with federated learning. *NPJ Digital Medicine* 3, 119. https://doi.org/10.1038/s41746-020-00323-1.

[3] Kairouz, P., McMahan, H. B., et al. 2021. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning* 14, 1–2, 1–210. https://doi.org/10.1561/2200000083.

[4] Li, T., Sahu, A. K., Talwalkar, A., and Smith, V. 2020. Federated optimization in heterogeneous networks. *In Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys 2020)*. 429–450.

[5] Li, Q., Wen, Z., Wu, Z., Hu, S., Wang, N., Li, Y., Liu, X., and He, B. 2019. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. arXiv preprint arXiv:1907.09693. https://doi.org/10.48550/arXiv.1907.09693.

[6] Zhou, Y., Ram, P., Salonidis, T., Baracaldo, N., Samulowitz, H., and Ludwig, H. 2021. FLoRA: Single-shot hyper-parameter optimization for federated learning. arXiv preprint arXiv:2112.08524. https://doi.org/10.48550/arXiv.2112.08524.

[7] Parra-Ullauri, J. M., Zhang, X., Bravalheri, A., Nejabati, R., and Simeonidou, D. 2023. Federated hyperparameter optimisation with Flower and Optuna. In *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*, 1209–1216. https://doi.org/10.1145/3555776.3577847.

[8] Abdullah Al Asif, Fan Zhou, Juan Pablo Munoz, and Ali Jannesari. 2026. FedAgent-HPO: Agentic hyperparameter optimization for personalized federated learning. *OpenReview*. https://openreview.net/forum?id=q78jTz1JDy.

[9] McMahan, B., Moore, E., Ramage, D., Hampson, S., and Aguera y Arcas, B. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (AISTATS 2017). PMLR, 1273–1282.

[10] Zawad, S., Yi, J., Zhang, M., Li, C., Yan, F., and He, Y. 2023. Demystifying hyperparameter optimization in federated learning. *OpenReview*. https://openreview.net/forum?id=m7S4NvprHVl.

[11] Pollard, T. J., Johnson, A. E. W., Raffa, J. D., Celi, L. A., Mark, R. G., and Badawi, O. 2018. The eICU Collaborative Research Database, a freely available multi-center database for critical care research. *Scientific Data* 5, 180178. https://doi.org/10.1038/sdata.2018.178.

[12] Beutel, D., Topal, T., Mathur, A., Qiu, X., Parcollet, T., de Gusmão, P. P. B., Lane, N. D., and Rieke, N. 2020. Flower: A friendly federated learning research framework. arXiv preprint arXiv:2007.14390. https://doi.org/10.48550/arXiv.2007.14390.

[13] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, 2623–2631. https://doi.org/10.1145/3292500.3330701.

[14] Prantzalos, J. 2026. *Convergence Dynamics of Agentic Hyperparameter Optimization in Federated Learning*. GitHub repository. Available at: https://github.com/umdnp/swe520capstone-agentic-hpo

APPENDIX

A. Agent Prompt and Output Schema

The agentic hyperparameter controller is implemented as a structured LLM-based decision system. The controller receives a structured JSON payload describing the current training state and returns a validated hyperparameter proposal.

A.1. Prompt Structure

The prompt consists of two components:

1. An instruction block defining the agent's role and strategy
2. A structured JSON payload describing current training state

The controller is guided by the following instruction block:

```
You are an expert federated learning hyperparameter
controller.

Each round, propose the next training hyperparameters.

You only see aggregated metrics and prior choices.

Goal: maximize roc_auc while keeping loss low and
training stable.

Do not overreact to one noisy round.

Early rounds may explore more; later rounds should prefer
smaller, conservative changes.

Treat penalty and learning-rate schedule changes as major
changes.

Prefer adjusting local_epochs or eta0 before changing
penalty or schedule.

Use constant learning rate only when there is clear
evidence that the current learning-rate approach is
underperforming.

Set exploit=1 only when you are intentionally keeping or
only slightly adjusting a configuration that has shown
stable or improving performance across multiple recent
rounds, and avoid exploit=1 too early in training.

Set exploit=0 when you are testing a meaningfully
different configuration.

If best_seen is provided, use it mainly in later rounds
as an anchor unless there is strong evidence to explore
elsewhere.
```

This instruction block is combined with a structured state payload of the form:

```
{
  "objective": {
    "maximize": "roc_auc",
    "minimize": "loss"
  },
  "round": t,
  "phase": "exploration" | "stabilization",
  "late_phase": boolean,
  "force_explore": boolean,
  "search_space": {
    "local_epochs": [3, 8],
    "penalty": ["l1", "l2", "elasticnet"],
    "sgd_learning_rate": ["optimal", "constant",
"adaptive"],
    "sgd_eta0": "if constant/adaptive: [1e-4, 1e-2]; if
optimal: 0.0",
    "exploit": [0, 1]
  },
  "current_hp": {...},
  "best_seen": {...},
  "history": [...],
  "history_summary": {...},
  "rules": [...]
}
```

The history_summary includes:

- Latest ROC-AUC and loss
- Rolling means over the last 3 and 5 rounds
- 5-round deltas for AUC and loss
- A plateau indicator defined as $|\Delta AUC_5| < 0.002$

A.2. Output Schema

The agent is required to return structured output conforming to the following schema:

```
local_epochs: integer in [3, 8]
penalty: one of {l1, l2, elasticnet}
sgd_learning_rate: one of {optimal, constant, adaptive}
sgd_eta0: float
exploit: 0 or 1
```

Additional constraints are enforced:

- If sgd_learning_rate = optimal, then sgd_eta0 = 0.0
- Otherwise, sgd_eta0 ∈ [1e-4, 1e-2]

All outputs are validated using a Pydantic schema prior to application.

A.3. Example Interaction

Example input (simplified state representation):

```
{
  "round": 8,
  "phase": "stabilization",
  "history_summary": {
    "auc_last": 0.612,
    "auc_delta_5": -0.001,
    "loss_last": 9.8,
    "plateau": true
  }
}
```

Example agent output:

```
{
  "local_epochs": 4,
  "penalty": "l2",
  "sgd_learning_rate": "adaptive",
  "sgd_eta0": 0.005,
  "exploit": 1
}
```

This structured prompting approach ensures that the agent operates within a constrained and reproducible decision space rather than relying on unconstrained or purely heuristic search.

A. Robustness and Fallback Behavior

To ensure stable execution during federated training, the controller includes multiple safeguards:

- Outputs are validated against schema and value constraints
- Invalid proposals trigger fallback to the previous config
- API or runtime failures also trigger fallback behavior
- Exploration is enforced in early rounds by setting exploit = 0

This design ensures that agent errors do not interrupt training or produce invalid configurations.

B. Best Configuration Tracking

During training, the system tracks the best observed hyperparameter configuration using a composite score:

$$\text{score} = \text{ROC-AUC} - 0.02 \times \text{loss}$$

This score is used only to identify the strongest configuration encountered during adaptive search. It is not used as a primary evaluation metric in reported results.